Singapore University of Technology and Design

60.001 Applied Deep Learning Project Report

Topic: Sign Language Word Classification

Name	Student ID
Charmaine Hong	1007066
Michelle Halim	1007039
Timothy Tang Long Zun	1006626
Muhammad Khairil Fahmi Bin Paroep	1006618

Github Link: https://github.com/Xephori/ADL.git

Google Drive with saved models and results:
https://drive.google.com/drive/folders/197ggC4-ApmuxbLZdNhSSSCJg2R8iSJeF?usp=drive_link

Name	Contribution
Timothy Tang Long Zun	Initial Dataset preprocessing
Charmaine Hong	Dataset preprocessing, Building and training models (Iteration 1-10), Evaluation of the trained models, Web app to show live demo Report
Michelle Halim	Presentation Slides
Muhammad Khairil Fahmi Bin Paroep	Presentation Slides

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Project scope 4
2. Problem Definition and Dataset 5
2.1 Task Definition 5
2.2 Dataset: WLASL 5
2.3 Data Preprocessing (Initial) 5
3. Related Work 6
3.1 Video-Based Sign Language Recognition 6
3.2 Temporal Modelling Strategies 6
3.3 Transfer Learning and Pretraining 6
3.4 Keypoint-Based Approaches 7
4. Proposed Architecture 7
4.1 CNN Backbone 7
4.2 Model A (CNN + Average Pooling) 8
4.3 Model B (CNN + Bidirectional LSTM) 8
4.4 Model C (CNN + BiLSTM + Temporal Attention) 8
4.5 Loss Function and Optimiser 9
5. Training Infrastructure and Key Optimisations 9
5.1 Frame Pre-Extraction 9
5.2 FastTensorLoader 9
5.3 Per-Epoch Logging 9
5.2 Default Training Configuration 9
6. Iterative Development 10
6.1 Iteration 1a, 100 Classes (Initial Attempt) 10
6.2 Iteration 1b, 20 Classes 10
6.3 Iteration 2, 10-Class Baseline 11
6.3.1 Data Preprocessing Change 11
6.4 Iteration 3, Transfer Learning: Model D (ResNet18) 12
6.4.1 Architecture Change: Model D 12
6.5 Iteration 4, CNN Pretraining on Own Video Frames 14
6.5.1 Architecture Change: Residual Blocks 14
6.5.2 Pretraining Strategy 15
6.6 Iteration 5, ASL Alphabet Pretraining 16
6.6.1 Data Preprocessing Addition 16
6.6.2 Three-Stage Pipeline 16
6.6.3 Results 16
6.6.4 Analysis 17
6.7 Iteration 6, Frozen CNN Experiment 17
6.8 Iterations 7–8, Learning Rate Tuning 18
6.9 Iteration 9, Multi-Seed Robustness 19
6.10 Iteration 10, MediaPipe Hand Keypoints 21

6.10.1 Architecture Change: Keypoint Model 21
7. Model Selection and Justification 22
7.1 Why Did We Not Use Validation Accuracy Alone 22
7.2 Tiered Evaluation Methodology 23
7.3 Final Model Selection 23
8. Final Model Evaluation 24
8.1 Final Model Architecture 24
8.2 Experimental Specifications 25
8.3 Final Metrics 25
8.4 Model Malfunctions 27
9. Key Findings and Analysis 27
9.1 CNN Feature Quality Is the Bottleneck 27
9.2 Simpler Models Work Better When Data Is Limited 27
9.3 Data Diversity Matters More Than Data Quantity 27
9.4 Validation Accuracy Is Unreliable on Small Datasets 28
9.5 Dataset Size was the Performance Ceiling 28
9.6 Summary of Hyperparameter Tuning 28
10. State-of-the-Art Comparison **29**
11. Conclusion and Future Work **29**
12. Reproducibility 30
12.1 Environment and Dependencies 30
12.2 Setup 30
12.3 Reproducing the Best Result (Iteration 5, Model A) 30
12.4 Training the Final Model from Scratch 30
12.5 Reproducing Report Figures 31
12.6 Reproducing Other Iterations 31
12.7 Running the Live Demo 31
13. References **31**

1. Introduction

Communication is fundamental to human connection, yet for over 70 million deaf individuals worldwide who rely on sign language as their primary mode of communication, the connection is consistently broken (World Federation of the Deaf, 2023). This is because, majority of the hearing population cannot understand sign language. Hence, this creates a barrier that limits deaf individuals' access to everyday social interactions.

While human interpreters partially bridge this gap, they are expensive, scarce, and unavailable in many everyday contexts. However, with the advancements in computer vision and deep learning it gives us an opportunity to address this. We can use image and video-based gesture recognition systems to automatically interpret hand signs and translate them into text, creating a connection with the deaf individuals.

Isolated word-level sign language recognition can be a solution to this problem. However, building a system that generalises reliably under real-world constraints is non-trivial. While large-scale benchmarks exist, access to those full datasets is not always guaranteed. Training robust deep learning models under such constraints requires careful architectural choices, pretraining strategies, and experimentation.These challenges brings us to our problem statement which drives this project:

“How might we design a deep learning pipeline for video-based ASL word classification which is able to generalise reliably to unseen signers given limited data and compute constraints?”

1.1 Project scope

We chose to focus specifically on classifying individual, isolated ASL signs from short video clips. Thus, this project does not address continuous sign language translation or sentence-level interpretation. The problem was scoped to 10 ASL word categories from the WLASL dataset, and most model components were built from scratch without borrowing any pretrained video recognition models. The only exceptions were a general-purpose image classifier (ResNet18) which was used as a comparison point in Iteration 3, and a dataset of static hand images used to pretrain the image-recognition component.

2. Problem Definition and Dataset

2.1 Task Definition

The classification task can be described as, given a short video clip of a person signing an ASL word, the system must identify which of 10 possible word classes is being signed. Each video is represented as a tensor of 16 evenly sampled RGB frames at 224 by 224 pixel resolution. The output is a single predicted class label drawn from the 10 target sign categories. The goal is to learn mapping from the input video space to the output label space using a parameterised deep neural network.

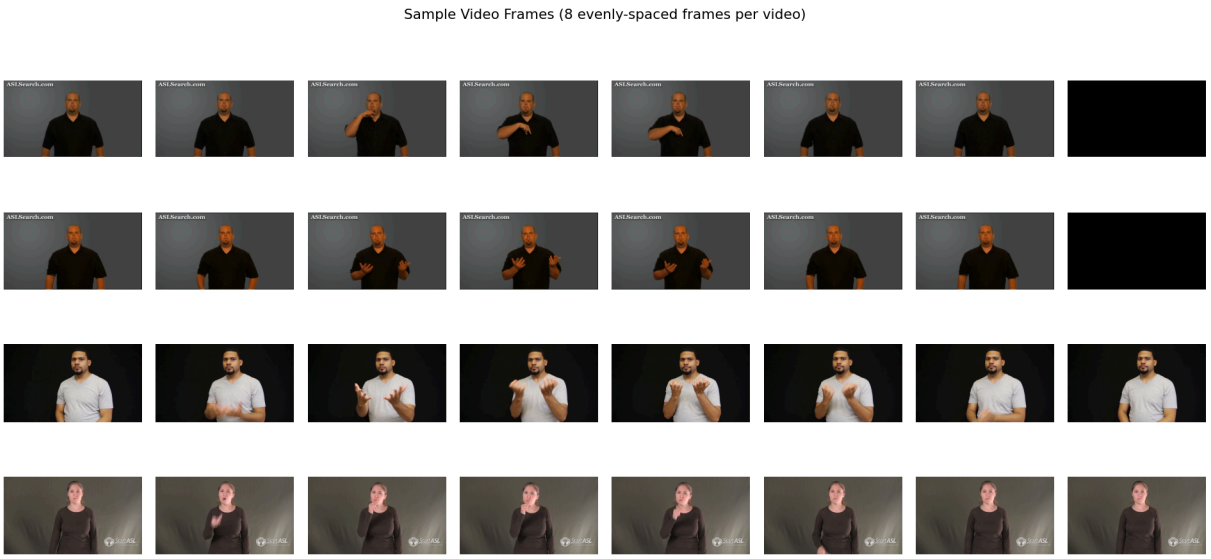

Figure 1: Sample Video Frames

2.2 Dataset: WLASL

The Word-Level American Sign Language (WLASL) dataset (Li et al., 2020) contains over 2,000 sign glosses performed by 100+ signers, sourced from instructional ASL videos. We began with the WLASL-100 subset. However, a significant portion of the original source videos had become unavailable by the time of download due to broken or expired links. This reduced our usable data to only 864 videos across 100 classes, which is approximately 8–9 videos per class, which was insufficient for reliable 100-class classification.

We therefore progressively reduced the number of classes to increase per-class sample representation, ultimately settling on 10 classes with 111 training videos (11 videos per class). The full details of this process are documented in Iterations 1a and 1b (Section 6.1–6.2).

The table below shows the final 10-class split used for the majority of experiments.

Split	Samples	Purpose
Train	111 (11/class)	Model training
Validation	20 (2/class)	Checkpoint selection
Test	18 (2/class)	Final evaluation

Table 1: Final 10 class dataset split

2.3 Data Preprocessing (Initial)

Before being fed into the model, each video underwent a series of standard preprocessing steps. Sixteen frames were evenly spaced across the video duration using OpenCV (cv2.VideoCapture). Each frame was resized to 224 by 224 pixels. Pixel values were normalised using standard ImageNet statistics with a mean of [0.485, 0.456, 0.406] and a standard deviation of [0.229, 0.224, 0.225] across the three RGB colour channels, to bring the input values into a stable range that the network can process efficiently. Random horizontal flipping was applied during training as a data augmentation technique. RandomRotation and ColorJitter augmentations were initially tested but removed due to extreme slowness on the computing cluster.

To address training speed, a frame pre-extraction pipeline was built using preextract_frames.py, which extracted all 16 frames per video once and saved them as pre-processed PyTorch tensor files (.pt). This reduced epoch time from over 20 minutes to under 60 seconds. In Iteration 5, a separate ASL Alphabet pretraining dataset of 87,000 images from Kaggle was introduced for CNN pretraining.

3. Related Work

Sign language recognition (SLR) has been studied extensively using both RGB video and skeleton-based approaches. We now look at key prior work and how our project relates to it.

3.1 Video-Based Sign Language Recognition

Early deep learning approaches to SLR adapted image classification architectures to video by processing individual frames independently. More recent methods use three-dimensional convolutional networks such as I3D (Carreira and Zisserman, 2017), which operate directly on spatiotemporal volumes to capture motion patterns across both space and time simultaneously. Video Transformers have since pushed accuracy to approximately 75% on the same benchmark by modelling long-range temporal dependencies through self-attention (Arnab et al., 2021). However, these architectures require substantially more training data and computational resources than what is available in this project, which is why we opted for a simpler and more modular CNN and LSTM-based design.

3.2 Temporal Modelling Strategies

There are several ways to combine information across video frames, from simple pooling that treats all frames equally regardless of order, to more complex methods such as recurrent networks and attention mechanisms. The Long Short-Term Memory (LSTM) network has been widely adopted for sequential modelling due to its gating mechanisms, which control how much information to remember, forget, and output at each timestep. Building on this, Bidirectional LSTM variants read the sequence in both directions, forwards and backwards, giving the model a fuller picture of what happens before and after each point in time. Bahdanau et al. (2015) further introduced an additive attention mechanism that allows a model to learn which parts of a sequence are most important, rather than treating all timesteps equally. We decided to incorporate all three of these approaches across our proposed architectures to measure how much each level of temporal complexity actually helps.

3.3 Transfer Learning and Pretraining

Transfer learning via ImageNet-pretrained backbones is a common practice in computer vision, where a model trained on a large general-purpose dataset is adapted to a new and smaller task. For sign language specifically, domain-specific pretraining on hand gesture datasets has shown particular promise (Koller et al., 2019), as the pretrained features are more directly relevant to the task. Building on these findings, we decided to explore a three-stage pretraining curriculum that progressively builds more task-relevant feature representations at each stage.

3.4 Keypoint-Based Approaches

An alternative to RGB-based methods is skeleton or keypoint extraction using tools such as MediaPipe Hands (Lugaresi et al., 2019). Rather than working with raw video pixels, keypoint representations track the positions of specific hand joints across time, removing appearance variation such as skin tone, clothing, and background while retaining the geometric and motion information about hand pose. We include a keypoint-based pipeline in our experiments to investigate whether the performance limits we observed were caused by the model overfitting to visual appearance, or whether they reflected a more fundamental constraint imposed by the size of the dataset.

4. Proposed Architecture

We designed our system as a modular two-stage pipeline. The first stage is a CNN feature extractor that independently processes each video frame into a 256-dimensional feature vector. The second stage is a temporal aggregation module that combines the 16 per-frame feature vectors into a single video-level representation, which is then passed to a classifier that produces the predicted sign label. Three temporal aggregation strategies of increasing complexity were proposed and compared.

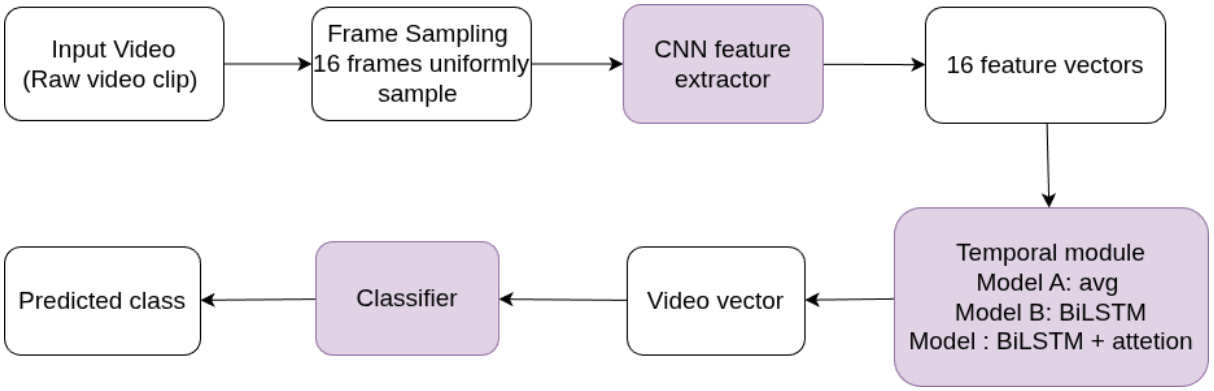

Figure 2: Shared architecture pipeline for Models A, B, and C

4.1 CNN Backbone

The initial CNN backbone is a custom four-layer convolutional network with channel sizes of 32, 64, 128, and 256. Each layer consists of a Conv2d layer with a 3 by 3 kernel, followed by BatchNorm2d, a ReLU activation function, and MaxPool2d with a 2 by 2 pooling window. After four convolutional blocks, adaptive average pooling reduces the spatial dimensions to 1 by 1, producing a 256-dimensional feature vector per frame. This CNN is entirely custom-built and not imported from any pretrained model library. In Iteration 4, the CNN was upgraded with residual skip connections to allow gradients to flow more stably through the deeper network.

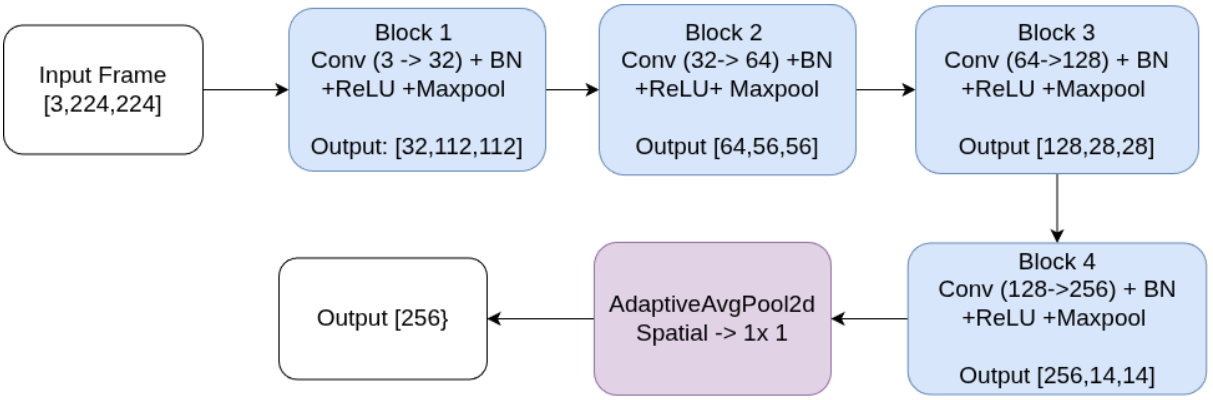

Figure 3: Original custom CNN backbone

4.2 Model A (CNN + Average Pooling)

Model A is the simplest temporal aggregation strategy. The 16 per-frame feature vectors produced by the CNN are averaged together element-wise to produce a single 256-dimensional video-level representation. This is similar to treating the video as an unordered bag of frames, with each frame contributing equally to the final summary. This averaged vector is then passed through a Dropout layer with dropout probability 0.3 and a fully connected linear classifier mapping from 256 dimensions to the number of output classes. Model A has no understanding of temporal order or motion dynamics.

4.3 Model B (CNN + Bidirectional LSTM)

Model B replaces average pooling with a Bidirectional LSTM with a hidden size of 128 and a single layer. The 16 CNN feature vectors are fed as a temporal sequence into the BiLSTM, which reads the sequence forward from frame 1 to frame 16, and simultaneously backward from frame 16 to frame 1. At each timestep, the LSTM applies its standard gating operations, the input gate, forget gate, cell state update, and output gate, to control how information flows through memory. The final hidden states from both the forward and backward passes are concatenated to form a 256-dimensional video-level representation that captures motion context from both temporal directions. This allows the model to distinguish between signs that look visually similar but move in opposite directions.

4.4 Model C (CNN + BiLSTM + Temporal Attention)

Model C extends Model B by retaining all 16 intermediate hidden states output by the BiLSTM, one per frame, and applying a learned temporal attention mechanism over them. The attention mechanism computes a score for each frame's hidden state by passing it through a small learned linear transformation followed by a tanh activation, producing an unnormalised relevance score. These scores are then normalised across all 16 frames using a softmax operation, ensuring they sum to one and can be interpreted as probabilities indicating each frame's importance. The final video-level representation is a weighted sum of all 16 hidden states, where frames assigned higher attention weights contribute more to the overall summary. This allows the model to focus on the most discriminative moments in the video rather than weighting all frames equally.

4.5 Loss Function and Optimiser

We use Cross-Entropy Loss with label smoothing set to 0.1. Label smoothing works by softening the training targets slightly, rather than training the model to assign 100% confidence to the correct class, it is trained to assign slightly less than 100%, which acts as a regulariser discouraging overconfident predictions and improving generalisation. The optimiser is Adam with β_1 set to 0.9 and β_2 set to 0.999. Gradient clipping with a maximum norm of 1.0 was applied via PyTorch's clip_grad_norm_ utility to prevent excessively large gradient updates from destabilising LSTM training.

5. Training Infrastructure and Key Optimisations

5.1 Frame Pre-Extraction

Video decoding was a primary bottleneck as each epoch decoded all videos with OpenCV, taking 20+ minutes per epoch. We created preextract_frames.py to extract and save all 16 frames per video as .pt tensors, reducing epoch time from 20 minutes to under 60 seconds.

5.2 FastTensorLoader

Even with pre-extracted tensors, PyTorch’s DataLoader was slow (10s per batch) due to tensor copying. We hence implemented FastTensorLoader a custom replacement that loads all tensors into one contiguous RAM block at startup, then slices directly. This helped batch latency dropped to near-instant.

5.3 Per-Epoch Logging

After an early session crash caused complete loss of training logs, we added CSV logging after every epoch to ensure partial results are preserved if sessions terminate unexpectedly.

5.2 Default Training Configuration

Parameter	Value
Optimiser	Adam (β_1=0.9, β_2=0.999)
Learning Rate	5×10^{-4} (unless stated)
Batch Size	8
Epochs	200 (unless stated)
Loss	CrossEntropyLoss + label smoothing (ε=0.1)
Dropout	0.3
Gradient Clipping	max_norm = 1.0
Augmentation	Random horizontal flip
Checkpoint (Iter 1–8)	Lowest validation loss
Checkpoint (Iter 9–10)	Highest validation accuracy
Early Stopping	Patience = 100
Scheduler	StepLR (step_size=50, γ=0.5)

Table 2: Default training hyperparameters.

6. Iterative Development

This section documents our complete experimental journey across 10 iterations. Each iteration was motivated by a specific hypothesis derived from the previous iteration's findings. The architecture evolved across iterations in response to empirical findings. Table 4 provides a summary of all the iterations.

Iter	Experiment	Models	Train Peak	Best Val	Test Acc	Key Finding
1a	100 classes	A,B,C	5-8%	2-5%	2%	Too many classes
1b	20 classes	A,B,C	15–19%	5–15%	8%	Still too many
2	10cls baseline	A,B,C	36–38%	20–25%	11.76%	CNN bottleneck
3	ResNet18 (D)	D	97–100%	75–80%	29.4/5.6%	Pretrained helps
4	Own frames	A,B,C	73–81%	30–40%	11–17%	Need diverse data
5	ASL pretrain	A,B,C	100%	45–55%	28–39%	Best: A=38.89%
6	Frozen CNN	A,B,C	37–44%	10–25%	11%	Too rigid
7	lr=5e-5	A,B,C	77–88%	5–25%	0–28%	LR too low
8	lr=1e-4	A	98%	35%	22%	Can't beat Iter 5
9	Multi-seed	A	100%	20–55%	17–39%	Seed-sensitive
10	MediaPipe	KP	81–100%	35–50%	33%	Dataset ceiling

Table 3 : Summary of all experimental iterations.

6.1 Iteration 1a, 100 Classes (Initial Attempt)

In our initial experiment we tried to train Models A, B, and C on all 100 sign categories in the WLASL-100 dataset, with approximately 864 training videos totalling roughly 8 to 9 videos per class. We thought that this data would be sufficient for our made from scratch CNN to begin learning discriminative sign features.

However, after we trained the model, we realised that all models remained stuck near random chance accuracy of approximately 1%. After analysing we realised that with 100 output classes and so few training examples per class, the final classification layer had far too many output neurons relative to the available data. Many signs also share similar hand shapes, making it impossible to learn reliable discriminative boundaries from only 8 samples per class. Hence, we reduce the number of classes to increase per-class representation.

6.2 Iteration 1b, 20 Classes

Reducing the task to the top 20 most frequent classes raised the random chance baseline to 5% and increased per-class training examples to approximately 10 to 13 videos. We tried testing a mixup augmentation technique which creates blended training examples by linearly interpolating between two video tensors and their corresponding label distributions using a Beta distribution with $\beta = 0.2$.

Sub-experiment	Model	Train Acc	Val Acc	Test Acc
20 class	A	19%	15%	8%
20cls + mixup	A	16%	10%	7%
20cls + mixup	B	15%	10%	7%
20cls + mixup	C	16%	10%	8%

Table 4: Iteration 1b. All models near chance level (5%).

However, all architectures still hovered near random chance, achieving training accuracy of 15 to 19% and validation accuracy of 5 to 15%. The mixup-blended frames produced visually meaningless hybrid sign images that provided no useful learning signal. Hence, we thought 20 classes remained too many for the CNN to handle, and reduced it to 10 classes.

6.3 Iteration 2, 10-Class Baseline

6.3.1 Data Preprocessing Change

With 10 classes, the random chance baseline rose to 10% and per-class training examples increased to approximately 11 videos. We added a top_n_classes parameter with label remapping to the dataloader to map the selected classes to contiguous indices from 0 to 9. The validation set shrank to 20 samples, so we set early stopping patience to 100 epochs to avoid premature termination on noisy validation metrics.

Model	Architecture	Train Acc	Val Acc	Test Acc
A	CNN + AvgPool	36%	25%	11.76%
B	CNN + BiLSTM	38%	20%	11.76%
C	CNN + BiLSTM + Attn	37%	20%	11.76%

Table 5: Iteration 2. All three models achieve identical 11.76% test accuracy.

Figure 4: Iteration 2 Training Curves

However, all three models A, B, and C achieved nearly identical test accuracy of 11.76%, barely above random chance. The fact that architecturally different models with fundamentally different temporal modelling strategies achieved identical performance was a key diagnostic for us. This is because, if temporal modelling provided any benefit, Models B and C would have outperformed Model A. The identical result across all three showed and proved to us that the CNN feature extractor could be the bottleneck as the CNN was learning to recognise specific individuals and backgrounds rather than discriminative hand shapes, producing features too weak for any temporal model to leverage.

6.4 Iteration 3, Transfer Learning: Model D (ResNet18)

6.4.1 Architecture Change: Model D

To address the CNN bottleneck, we tried to replace the custom CNN with a pretrained ResNet18 backbone, which had already been trained on ImageNet with 1.4 million general-purpose images. Layers 1 and 2 of ResNet18 were frozen to preserve the low-level pretrained features, while Layers 3 and 4 were fine-tuned during training. The temporal module used was the same BiLSTM with temporal attention as Model C.

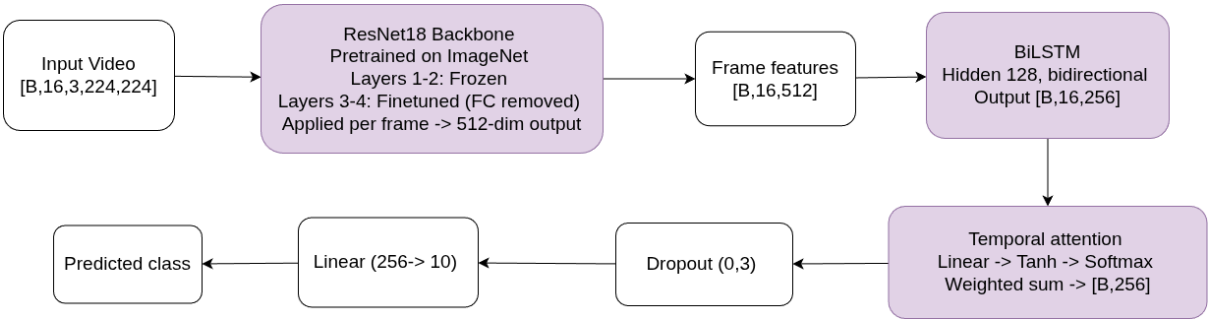

Figure 5: Model D architecture

Variant	Train Acc	Val Acc	Test Acc
Model D v1 (100 ep, lr=0.001)	93.5%	75%	29.41%
Model D v2 (200 ep, lr=0.0005)	100%	75%	5.56%

Table 6: Iteration 3. v1 is 2.5× better than baseline and v2 collapses despite same val accuracy.

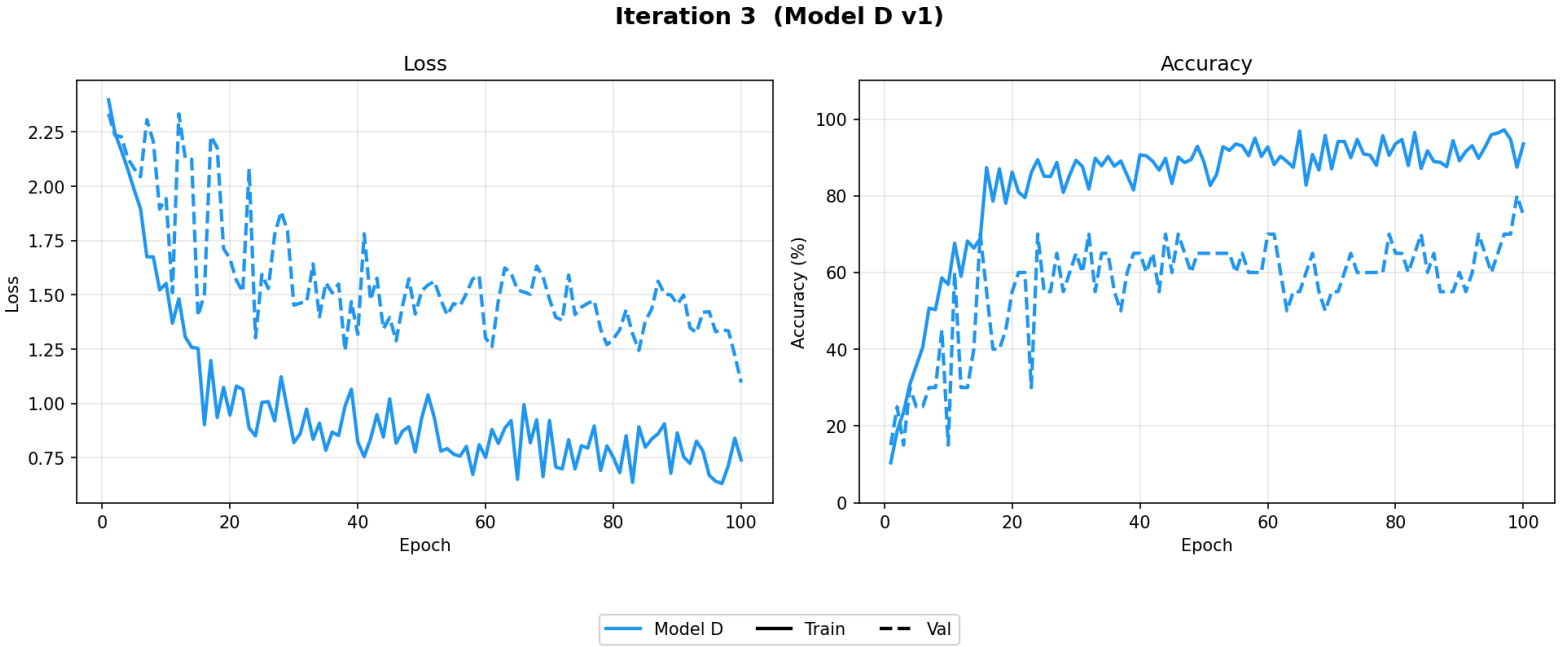

Figure 6: Iteration 3 Model D v1 Training curve

We tested two different versions. The first version, we trained it for 100 epochs with a learning rate of 0.001, and it achieved 29.41% test accuracy which is a 2.5-times improvement over the from-scratch baseline. For the second version, we trained it for 200 epochs at a lower learning rate of 0.0005. However, it only got a 5.56% test accuracy despite showing almost identical validation accuracy of 75 to 80% to version 1. This led us to understand that validation accuracy on only 20 samples is quite unreliable as the extended training caused deeper overfitting to the training set that the small validation set completely failed to detect.

6.5 Iteration 4, CNN Pretraining on Own Video Frames

6.5.1 Architecture Change: Residual Blocks

Then, we tried to upgrade the CNN backbone with residual skip connections, increasing from 4 to 8 convolutional layers organised as 4 residual blocks. Each residual block contains two Conv2d layers with BatchNorm2d and ReLU, plus a skip connection that adds the block's input directly to its output before a final ReLU. This shortcut pathway can allow gradients to flow directly through the skip connection during backpropagation, stabilising training of the deeper network and preventing the vanishing gradient problem. Channel sizes was remained at 32, 64, 128, and 256 but each block now contains two convolutional layers instead of one.

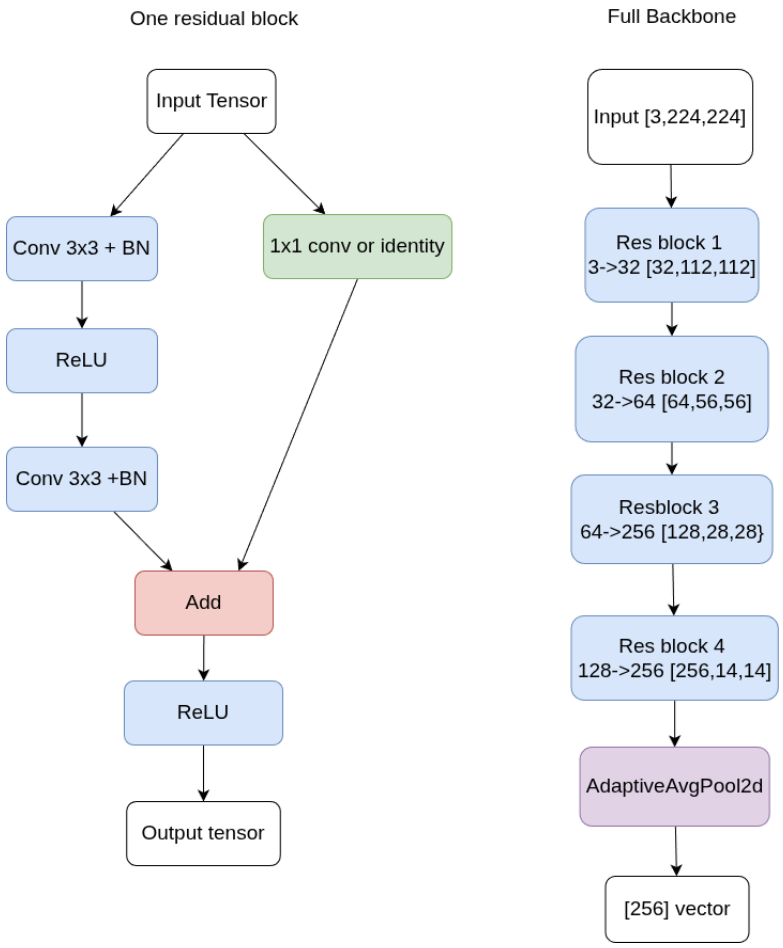

Figure 7: Updated CNN backbone with residual blocks, used from Iteration 4 onwards

6.5.2 Pretraining Strategy

Instead of training the model on full video clips (which gives 111 training examples per epoch), we also tried a different approach, which is to treat every individual frame from every video as its own separate training example. Since each video has 16 frames and there are 111 videos, this gives 1,776 individual images to train on per epoch, which is 16 times more training signals than before. The idea was that more training signals would help the model learn faster and better.

However, the improvement was very small. The reason is that the 1,776 frames are not actually 1,776 different and unique examples. They all came from the same 111 videos, and consecutive frames within the same video look almost exactly the same. So the model was seeing the same small set of people and backgrounds repeated over and over, just slightly different versions of each other. This did not give the model any genuinely new information to learn from, which is why the improvement was so limited.

Model	No Pretrain	Own Frames	Change
A	11.76%	11.11%	-0.65%
B	11.76%	16.67%	+4.91%
C	11.76%	16.67%	+4.91%

Table 7: Iteration 4 test accuracy. B/C outperform A

Figure 8: Iteration 4 Training Curves

6.6 Iteration 5, ASL Alphabet Pretraining

6.6.1 Data Preprocessing Addition

Moving forward, we introduced a new external pretraining dataset which was the ASL Alphabet dataset from Kaggle, containing 87,000 images of static A-to-Z hand signs contributed by hundreds of different signers. Up to 1,000 images per class were sampled, totalling 29,000 images across 29 classes. We preloaded the images into RAM for fast training. The CNN pretraining achieved 99.98% validation accuracy on this dataset, confirming the CNN had learned high-quality, generalisable hand shape features.

6.6.2 Three-Stage Pipeline

The training then followed a three-stage pipeline. In Stage 1, the CNN was pretrained on the 29,000 ASL Alphabet images for 29-class classification. In Stage 2, the pretrained CNN was fine-tuned on the 1,776 individual video frames from the 10-class training set. In Stage 3, the fine-tuned CNN was loaded into Models A, B, and C, and the full video-level pipeline was trained end-to-end for 200 epochs with a learning rate of 0.0005.

6.6.3 Results

Model	Architecture	Val Acc	Train Acc	Test Acc	Val-Test Gap
A	CNN + AvgPool	45%	100%	38.89%	6%
B	CNN + BiLSTM	50%	99.1%	27.78%	22%
C	CNN + BiLSTM + Attn	55%	100%	33.33%	22%

Table 8: Iteration 5. Model A achieves highest test accuracy despite lowest val accuracy

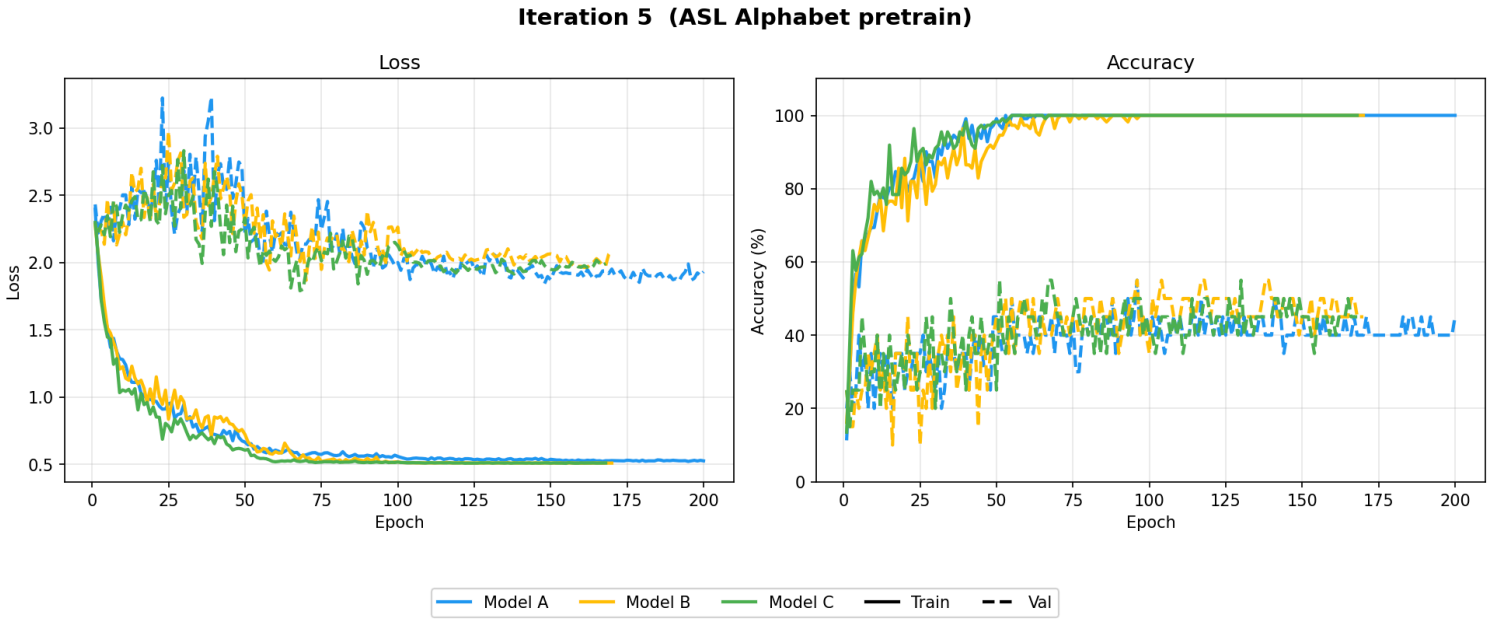

Figure 9: Iteration 5 Training Curves

6.6.4 Analysis

The results was a major improvement. Model A achieved 38.89% test accuracy, Model C achieved 33.33%, and Model B achieved 27.78%. Interestingly, Model A, the simplest architecture, achieved the highest test accuracy despite having the lowest validation accuracy of the three (45% versus 50% and 55%). This reversed the ranking observed in Iteration 4, where the more complex temporal models was performing better.

After analysing it we realised and learnt that the reason for this was because when CNN features are rich and discriminative, the BiLSTM's additional 2.6 million parameters cause overfitting to the signer-specific movement patterns of the 111 training individuals rather than learning generalisable temporal dynamics. With strong CNN features, simple average pooling across frames already captures enough information for effective classification. Furthermore, the custom CNN pretrained on 29,000 domain-specific hand images outperformed the ResNet18 pretrained on 1.4 million general-purpose ImageNet images (38.89% versus 29.41%), showing that training on relevant data matters more than simply having a large amount of it.

CNN Quality	Model A	Models B/C	Winner	Why
Weak (Iter 4)	11.11%	16.67%	B/C	LSTM compensates for weak CNN
Strong (Iter 5)	38.89%	28–33%	A	Rich features make LSTM unnecessary LSTM overfits

Table 9: The reversal pattern.

6.7 Iteration 6, Frozen CNN Experiment

In an attempt to improve the model from Iteration 5, we had an hypothesis that the full end-to-end fine-tuning destroys the pretrained CNN features, and that freezing the CNN entirely would better preserve the hand-shape representations learned during ASL Alphabet pretraining. Thus, we went ahead to try and see if this hypothesis was right. But after training we realised that this approach actually was backfiring on us. Model A's test accuracy dropped from 38.89% to 11.11%.

After analysing we realised that with the CNN fully frozen, Model A's only trainable component was a single linear classification layer with fewer than 3,000 parameters which was far too few to bridge the domain gap between static letter classification and dynamic word-level sign classification.

Models B and C fared marginally better because their BiLSTM components had more trainable parameters to compensate, but all models performed substantially worse than in Iteration 5. Hence, we learnt that some degree of CNN fine-tuning is necessary to adapt the pretrained features to the video-level classification task.

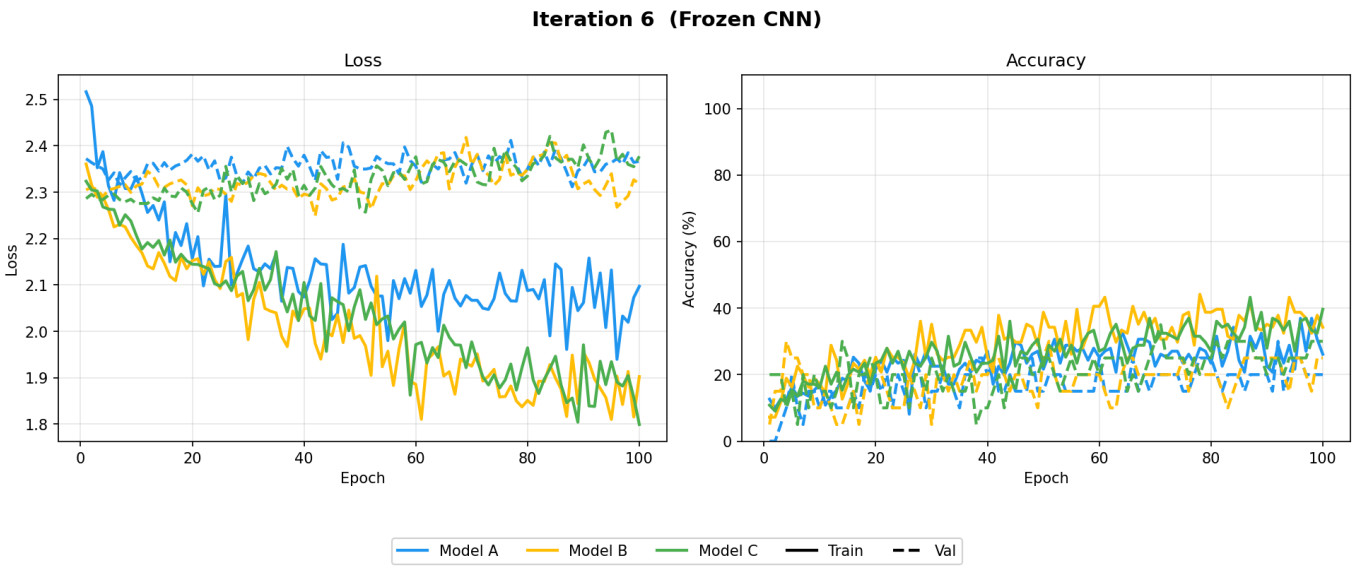

Figure 10: Iteration 6 Training Curves

6.8 Iterations 7–8, Learning Rate Tuning

Thus, after the not as good results in Iteration 6, we decided to go back to our old model in Iteration 5 and analysed further. From there, we started to hypothesise and explore whether a lower learning rate would allow the CNN to fine-tune more gently.

In Iteration 7, we used a learning rate ten times smaller than the default (5×10^{-5}) and Model A failed entirely achieving 0% test accuracy. The cause was a checkpoint selection failure with such a low learning rate, Model A had barely begun training by epoch 5, and validation loss dipped slightly due to random noise, hence the checkpoint saved an untrained model.

In Iteration 8, we used a learning rate five times smaller than default (1×10^{-4}) on just Model A and it achieved 22.22% test accuracy and recorded the lowest validation loss ever observed across all experiments. Hence, we did not go on to train Model B and C and learnt that the default learning rate of 0.0005 remained the best choice.

Iter	LR	Epochs	Model A Test	Model B Test	Model C Test
5 (ref)	5×10^{-4}	200	38.89%	27.78%	33.33%
7	5×10^{-5}	150	0%	16.67%	27.78%
8	1×10^{-4}	150	22.22%	—	—

Table 10: Learning rate comparison.

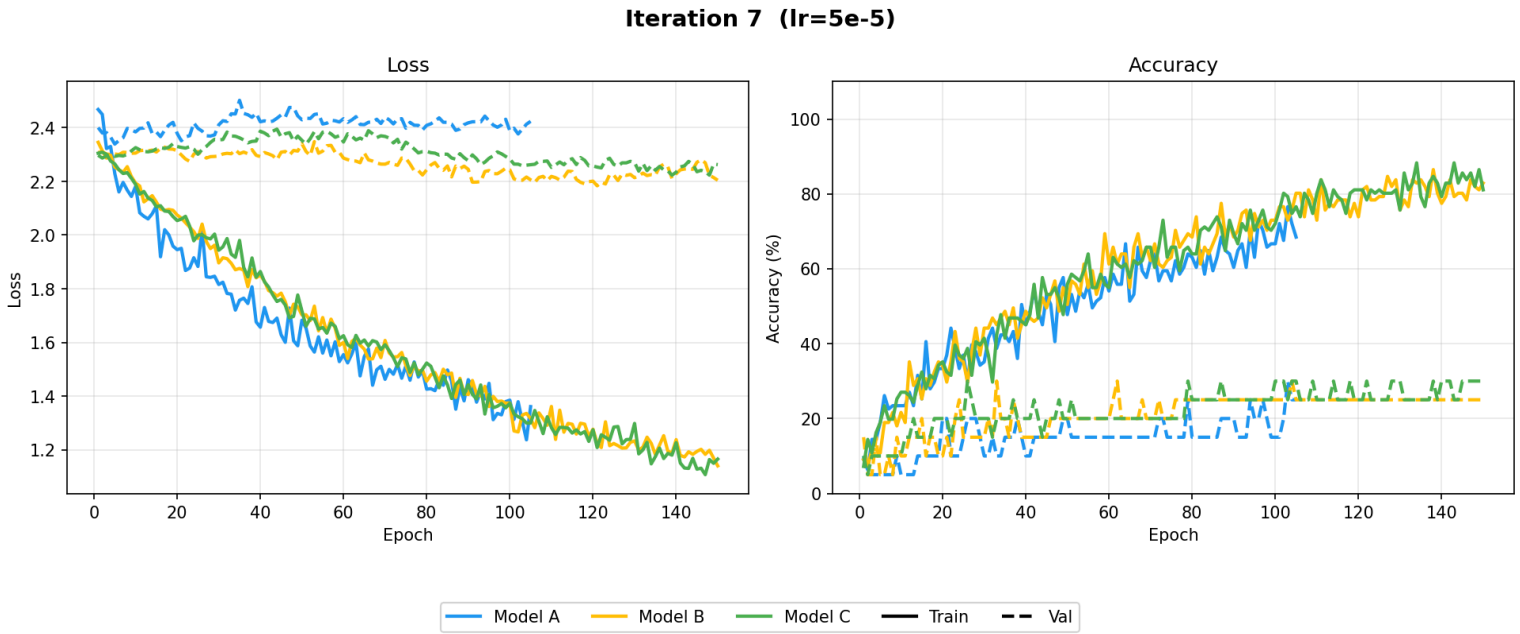

Figure 11: Iteration 7 Training Curves

6.9 Iteration 9, Multi-Seed Robustness

To further analyse, we also just wanted to check if the 38.89% result from Iteration 5 was genuinely representative or simply a lucky outcome of a particular random seed.

Hence, we used the same Iteration 5 setup and repeated it with four different random seeds. 42 (the original), 1, 7, and 123.

Seed	Val Acc	Train Acc	Test Acc
42 (Iter 5)	45%	100%	38.89%
1	50%	100%	27.78%
7	55%	100%	27.78%
123	20%	25.2%	16.67%

Table 11: Multi-seed results. 22-point swing in test accuracy across seeds.

Test accuracy ranged from 16.67% to 38.89%, a swing of over 22 percentage points across seeds determined solely by random initialisation. Excluding seed 123, which failed to converge, the mean test accuracy across the remaining three seeds was 31.5% with a standard deviation of 6.4%. Including all four seeds, the mean was 27.8% with a standard deviation of 9.2%. However, the original seed 42 still had the highest test accuracy.

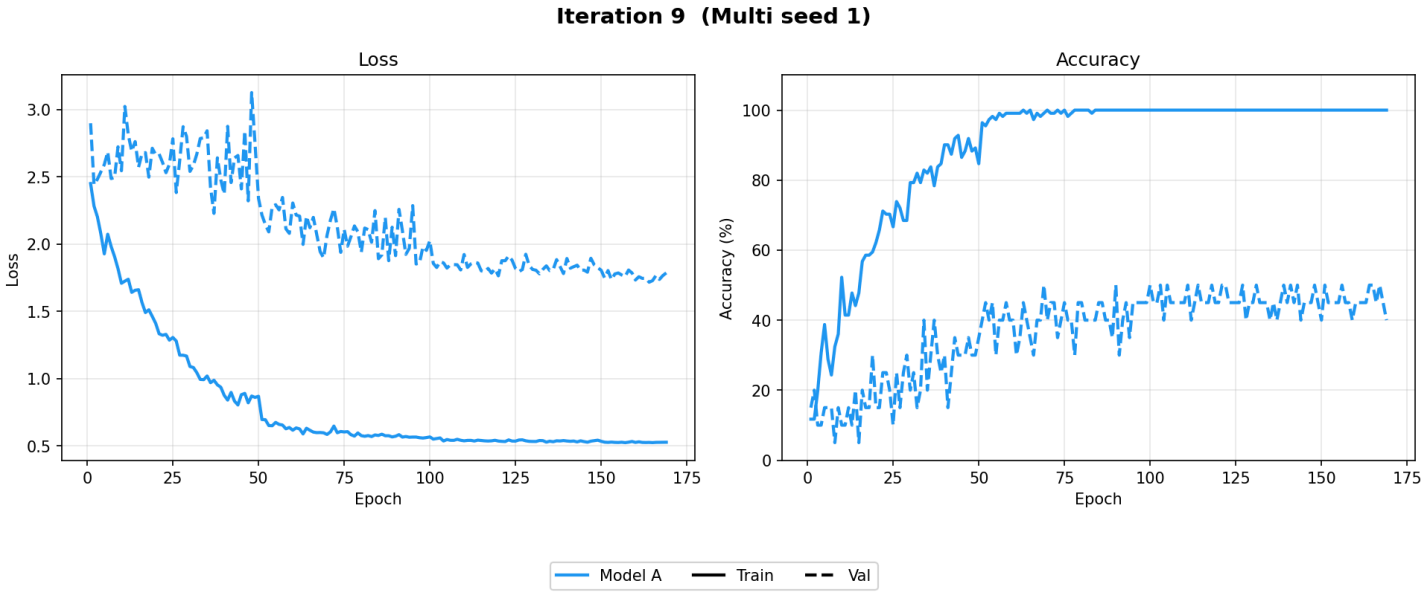

Figure 12: Iteration 9, Multi Seed 1 Training Curves

6.10 Iteration 10, MediaPipe Hand Keypoints

6.10.1 Architecture Change: Keypoint Model

We then moved on to try one last idea which was to replace raw RGB video frames with skeletal hand keypoint representations extracted using Google MediaPipe Hands. MediaPipe detects 21 hand landmarks per frame, the joints of each finger and the wrist, each providing three-dimensional (x, y, z) coordinates, resulting in 63 features per frame.

Landmarks were normalised by subtracting the wrist coordinate, providing translation invariance across different hand positions in the frame. A BiLSTM with hidden size 128, 2 layers, and approximately 595,000 total parameters was trained to classify the resulting 16-by-63 temporal sequences.

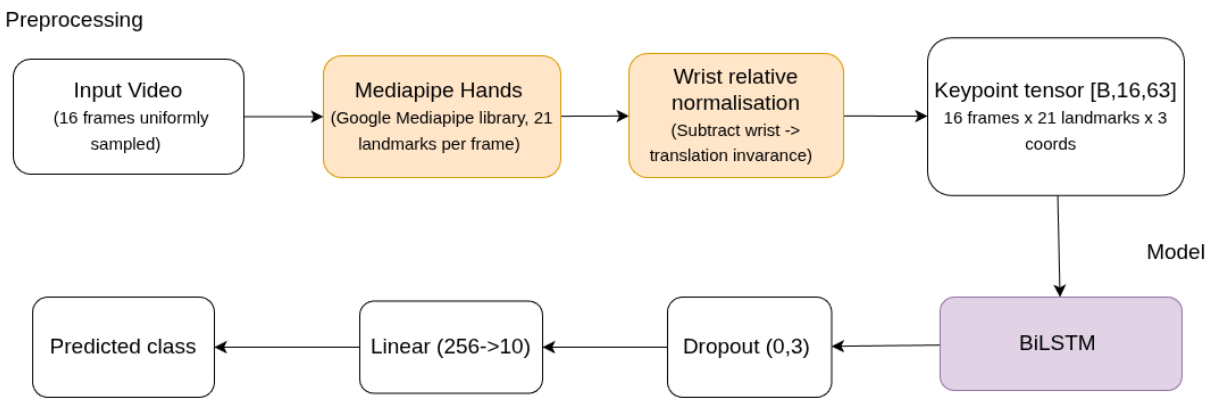

Figure 13: MediaPipe Keypoint pipeline

Variant	Val Acc	Train Acc	Test Acc
v1 (baseline)	50%	100%	33.33%
v2 (noise aug)	35%	81%	33.33%

Table 12: MediaPipe results. Same overfitting pattern.

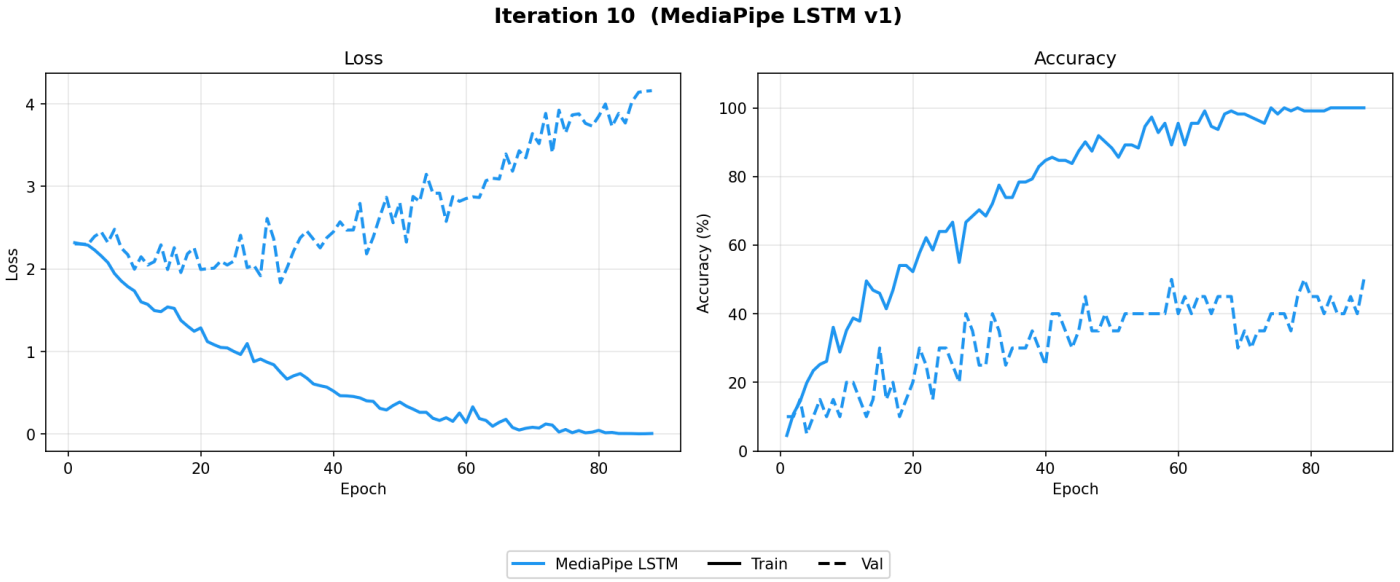

Figure 14: Mediapipe LSTM Version 1 Training Curves

For the initial baseline keypoint classification, it achieved 33.33% test accuracy which was still low. Hence we tried to add noise augmentation, which is adding small Gaussian noise to the keypoint coordinates during training to reduce overfitting. However, it also ended up achieving 33.33% test accuracy. We noticed however, that the training accuracy was lower at 81%, which indicates slightly better generalisation. The fact that this input representation converged to nearly the same performance ceiling as the RGB-based approaches even though this approach was supposed to remove all appearance information and retaining only geometric motion data strongly confirms to us that the performance limit is determined by the dataset size rather than by any particular model design or input representation. Hence, we decided to stop training at Iteration 10.

7. Model Selection and Justification

After these 10 iterations, we will now choose the final model that we will select. This section will show how we compared the different models and what basis did we compare them.

7.1 Why Did We Not Use Validation Accuracy Alone

Standard machine learning practice uses a held-out validation set for model selection. However, the validation set in this project contains only 20 samples, approximately 2 per class. At this scale, a single correct or incorrect prediction shifts measured accuracy by 5 percentage points, a level of noise that makes reliable model comparison nearly impossible. Evidence of this unreliability appeared repeatedly throughout the experiments. In Iteration 3, two model variants showed nearly identical validation accuracy of 75 to 80% yet differed by 24 percentage points in test accuracy. In Iteration 9, the same architecture with different random seeds produced validation accuracy ranging from 20% to 55%. In Iteration 8, the checkpoint with the lowest validation loss ever recorded across all experiments produced 22.22% test accuracy.

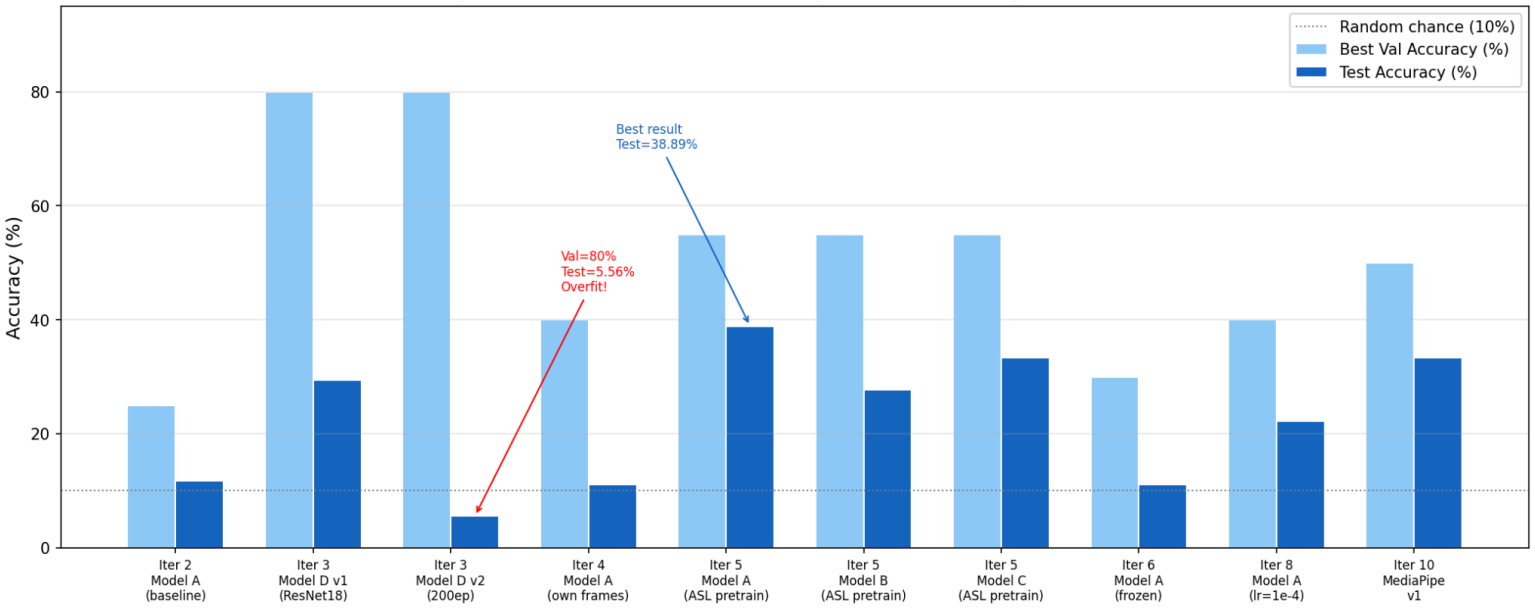

Figure 15: Validation and Test Accuracy comparison graph

7.2 Tiered Evaluation Methodology

Hence, given the unreliability of any single metric on small datasets, a tiered evaluation methodology was developed. Within a training run, the checkpoint with the lowest validation loss was saved as the best model. Across different training runs and architectures, validation accuracy was used as a coarse first filter to eliminate clearly weak candidates. More detailed metrics, including macro F1-score, per-class precision and recall, and confusion matrix analysis, were then used to differentiate between top candidates.

The test accuracy was the final and most important measure used to judge how good a model is. This is because, we only ran each model against the test set once, because if the test set was checked repeatedly and adjusted based on those results, the test set essentially becomes part of the training process, meaning the results are no longer a fair reflection of how the model would perform on completely unseen data. By only checking it once, the evaluation stays clean and unbiased.

7.3 Final Model Selection

	Iter 5 Model A	**Iter 5 Model C**	**Iter 3 Model D v1**	**Iter 10 MediaPipe**
Val Acc	45%	55%	75%	50%
Test Acc	38.89%	33.33%	29.41%	33.33%
Val-Test Gap	6%	22%	46%	17%
Overfitting	Severe	Severe	Severe	Severe

Table 13: Final comparison of top candidates.

Our final selected model was Iteration 5, Model A. The custom CNN with residual blocks pretrained on ASL Alphabet images, combined with simple temporal average pooling. This model achieved the highest test accuracy of any experiment at 38.89%, with the smallest validation-to-test gap of 6 percentage points across all top candidates. Hence it indicates the most consistent generalisation behaviour.

8. Final Model Evaluation

8.1 Final Model Architecture

The final selected model we selected has a two-stage pipeline consisting of a custom CNN feature extractor with residual blocks and a temporal average pooling classifier. The full architecture is described in Sections 4.1 and 4.2, and the three-stage pretraining pipeline is described in Section 6.6.2. In short, the CNN processes each of the 16 input frames independently into a 256-dimensional feature vector. These are then averaged element-wise into a single video-level representation, which passes through a Dropout layer (p = 0.3) and a linear classifier mapping to 10 output classes.

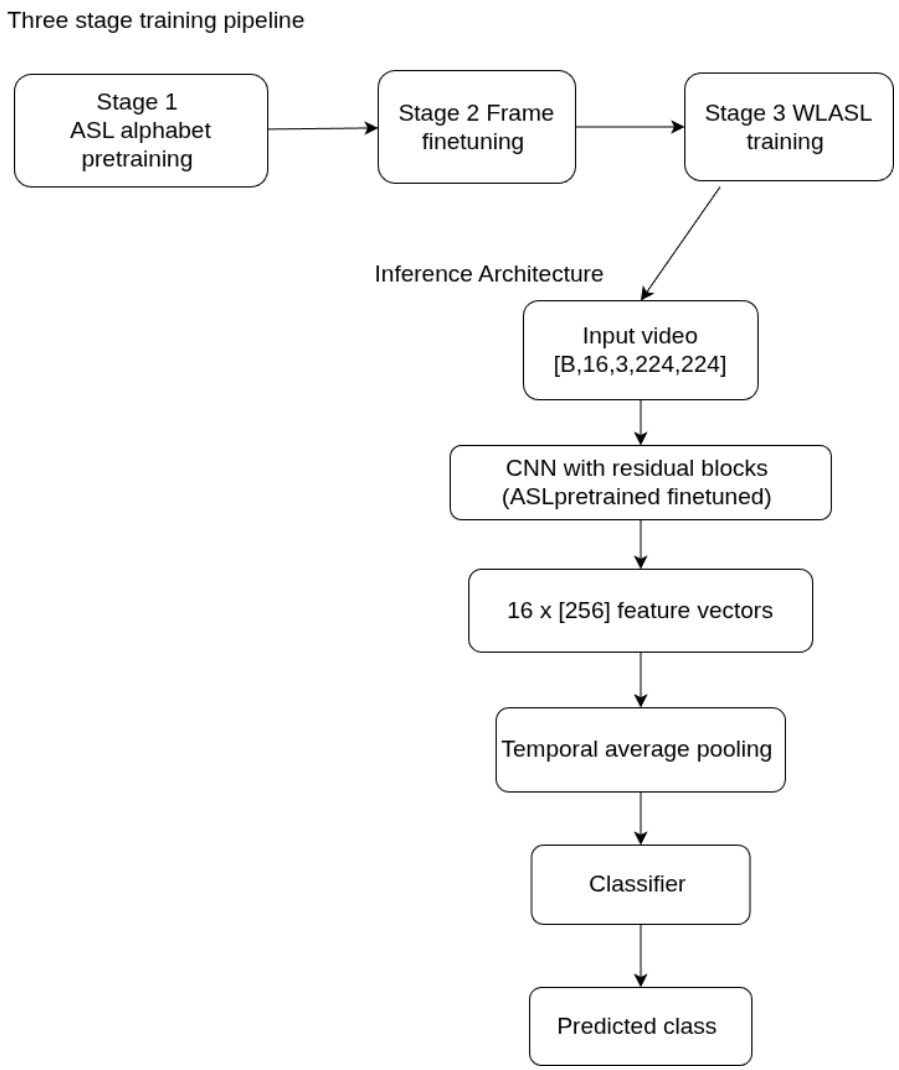

Figure 21: Final Model Architecture

8.2 Experimental Specifications

The table below shows the complete experimental specifications for the final model.

Specification	Value
Input format	Video 16 RGB frames at 224×224, ImageNet-normalised
Output format	10-class probability vector (softmax)
Training set	111 videos, 10 classes (11 per class)
Validation set	20 videos (2 per class)
Test set	18 videos (2 per class), held out, never used for model selection
Training time (Iter 5)	20 minutes for 200 epochs (with pre-extracted frames)
CNN pretraining time	10 minutes for 15 epochs on 29K ASL Alphabet images
Baseline comparison	Random chance = 10% (1/10 classes)
Evaluation metrics	Top-1 accuracy, Top-5 accuracy, Macro F1, Precision, Recall, per-class breakdown, confusion matrix

Table 14: Complete experimental specifications for the final model.

8.3 Final Metrics

Metric	Value
Top-1 Accuracy	38.89% (7/18)
Top-5 Accuracy	61.11% (11/18)
Macro F1-Score	0.34
Macro Precision	0.34
Macro Recall	0.40
Multi-seed Mean ± Std	27.8% ± 9.2% (n=4 seeds)

Table 15: Final metrics on 18-sample test set.

Per-Class Metrics:

Sign	Precision	Recall	F1
accident	0.00	0.00	0.00
before	0.25	0.50	0.33
computer	0.50	0.50	0.50
cool	0.33	0.50	0.40
cousin	0.00	0.00	0.00
drink	1.00	0.50	0.67
give	0.33	1.00	0.50
go	0.50	0.50	0.50
inform	0.00	0.00	0.00
thin	0.50	0.50	0.50

Table 16: Per-class metrics.

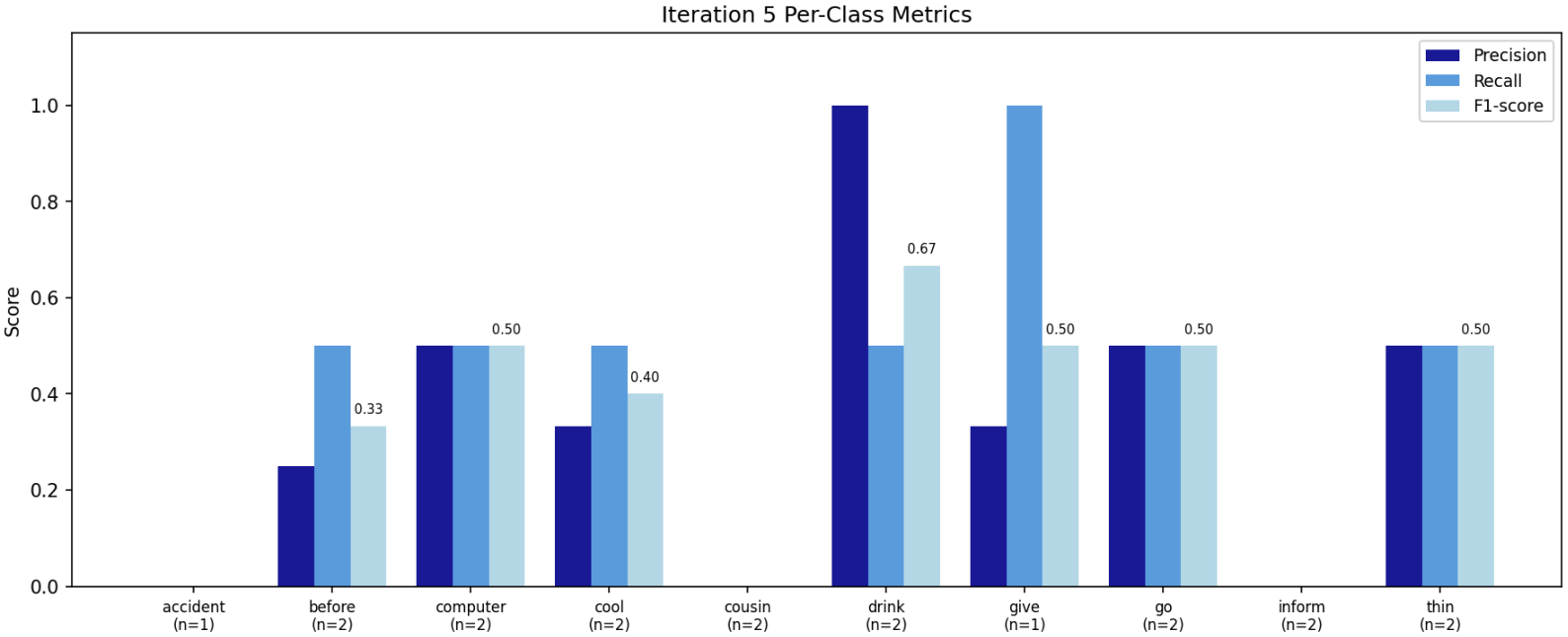

Figure 16: Iteration 5 Per Class Metrics Bar Graph

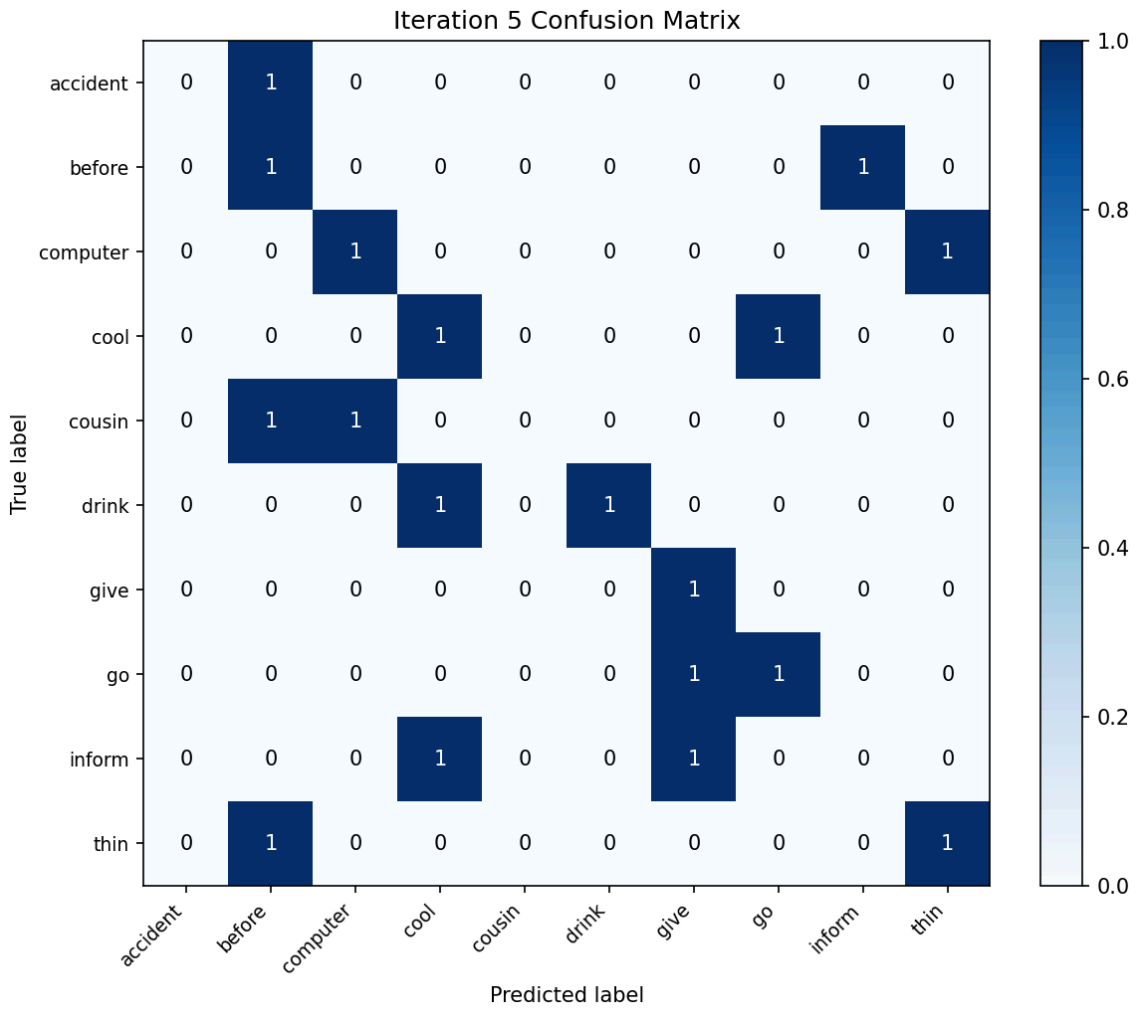

Figure 17: Iteration 5 Confusion Matrix

8.4 Model Malfunctions

Three classes achieved an F1-score of zero: accident, cousin, and inform. These signs involve subtle finger movements and fine-grained hand configurations that are difficult to distinguish at 224 by 224 pixel resolution with only 16 sampled frames per clip. As there were only approximately 11 training examples per class, the model did not have sufficient exposure to the variation of these visually similar signs. The best-performing classes were drink (F1 of 0.67, perfect precision) and give (100% recall), both of which have distinctive hand shapes well-captured by the ASL Alphabet pretrained features.

9. Key Findings and Analysis

9.1 CNN Feature Quality Is the Bottleneck

The most important factor across all 10 experiments was the quality of the spatial features produced by the CNN feature extractor. As the CNN features improved, so did the test accuracy, going from 11.76% when features were learned from scratch on limited sign word videos, to 29.41% when features were transferred from a general-purpose ImageNet-pretrained model, and finally to 38.89% when the CNN was specifically pretrained on a large and diverse dataset of hand images. In comparison, changes to the temporal modelling strategy, learning rate, and regularisation all had much smaller effects on the final performance. This shows that getting the CNN to produce good, meaningful features is the single most important thing to focus on in this kind of task.

9.2 Simpler Models Work Better When Data Is Limited

When the CNN features were strong, the simplest temporal aggregation strategy of average pooling outperformed both the BiLSTM and the BiLSTM with temporal attention. This is explained through the bias-variance tradeoff. When training data is limited, models with more parameters tend to overfit, meaning they memorise the training data rather than learning patterns that generalise to new examples. The BiLSTM's additional 2.6 million parameters gave it too much capacity, causing it to memorise the specific movement habits of the 111 training individuals rather than learning generalisable temporal dynamics. When the CNN features were weak, as seen in Iteration 4, the BiLSTM did provide a modest benefit by partially compensating for the poor spatial representations. However, this advantage disappeared once the CNN produced sufficiently rich and discriminative features.

9.3 Data Diversity Matters More Than Data Quantity

Iteration 4 showed that increasing the number of training signals by 16 times, from 111 video-level samples to 1,776 frame-level samples, produced only modest improvement. This is because consecutive frames within the same video are visually nearly identical, meaning the model was seeing the same limited variation repeated many times without encountering genuinely new information. By contrast, using 29,000 ASL Alphabet images drawn from hundreds of different people in varied settings produced a threefold improvement in test accuracy. This confirms that the diversity of the pretraining data across different signers, skin tones, backgrounds, and lighting conditions is far more important than simply having a larger number of training examples.

9.4 Validation Accuracy Is Unreliable on Small Datasets

With only 20 validation samples (2 per class), a single prediction swing shifts class accuracy by 50%, making validation metrics unreliable alone. To address this, we used a tiered evaluation approach: validation loss for checkpoint selection, validation accuracy as a coarse filter, F1-scores and confusion matrices for deeper comparison, and test accuracy as the final measure

9.5 Dataset Size was the Performance Ceiling

Despite very different architectures, a large 14.4M-parameter pretrained model, RGB-based approaches, and a keypoint-based model, all converged to a similar accuracy range of around 33–39%. This suggests the performance ceiling is set by dataset size, not modelling choices. With only about 11 training examples per class, additional compute or tuning can only help incrementally.

9.6 Summary of Hyperparameter Tuning

Across all experiments, the most impactful factors were the number of target classes and CNN pretraining strategy. Learning rate had a moderate effect, with 5×10^{-4} performing best. Architecture choice (Models A–C) showed conditional effects depending on CNN feature quality. Regularisation improved training stability but not accuracy significantly. Random seed variance was notable, highlighting the need for multi-seed evaluation on small datasets

Hyperparameter	Values Tested	Best Value	Iteration(s)
Number of classes	100, 20, 10	10	1a, 1b, 2
Architecture	A (AvgPool), B (BiLSTM), C (BiLSTM+Attn), D (ResNet18+BiLSTM+Attn), KP (MediaPipe+BiLSTM)	A (with strong CNN)	2–5, 10
CNN pretraining	None, own frames (1.7K), ASL Alphabet (29K), ImageNet (ResNet18)	ASL Alphabet	2–5
CNN fine-tuning	Full fine-tune, fully frozen	Full fine-tune	5, 6
Learning rate	5×10^{-4}, 1×10^{-4}, 5×10^{-5}, 1×10^{-3}	5×10^{-4}	5, 7, 8, 3
Epochs	100, 150, 200	200	3, 5, 7–8
Data augmentation	None, horizontal flip, mixup (α=0.2), Gaussian noise (keypoints)	Horizontal flip	1b, 2–9, 10
Random seed	42, 1, 7, 123	42 (best), but seed-sensitive	9
Checkpoint strategy	Best val loss, best val accuracy	Best val accuracy (Iter 9–10)	All
Input representation	RGB frames, MediaPipe hand keypoints	RGB frames	2–9, 10

Table 17: Summary of all hyperparameters explored across 10 iterations.

10. Interpretability via Attention Visualisation

Beyond accuracy, knowing which parts of a video a model focuses on can reveal what it has actually learned. Sign language is ideal for this because each sign has a clear structure. A start, a core movement, and an end. We used Model C (CNN + BiLSTM + Temporal Attention) for this analysis since it's the only model that directly outputs per-frame attention weights, making its focus interpretable.

10.1 Attention Extraction

The attention mechanism in Model C produces a probability distribution α over the 16 frames through a softmax operation, where each α_i represents the relative importance of frame i to the final prediction. For each test video, we performed a forward pass through Model C, extracted α from the attention layer, and plotted it as a bar chart beneath the 16 sampled frames. This produces a visual mapping between what the model saw and what it decided mattered.

10.2 Per-Sample Attention Patterns

To examine how the attention mechanism behaves on individual videos, we visualised attention weights for both correctly classified and misclassified samples from the test set.

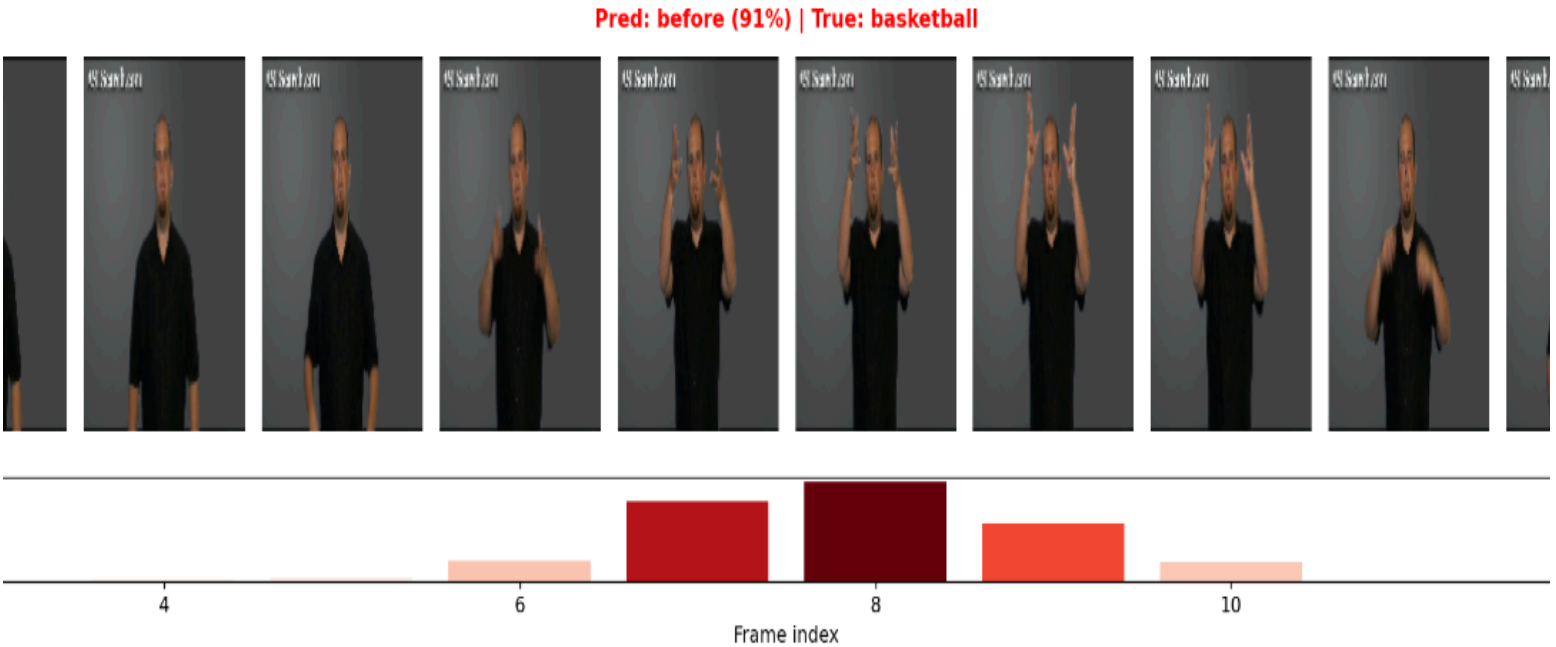

Figure 18: Attention on misclassified "basketball", incorrectly predicted as "before" with 91% confidence. The attention forms a clear peak at frame 8 but on a non-discriminative frame, causing the model to match the handshape to a similar-looking sign from a different class.

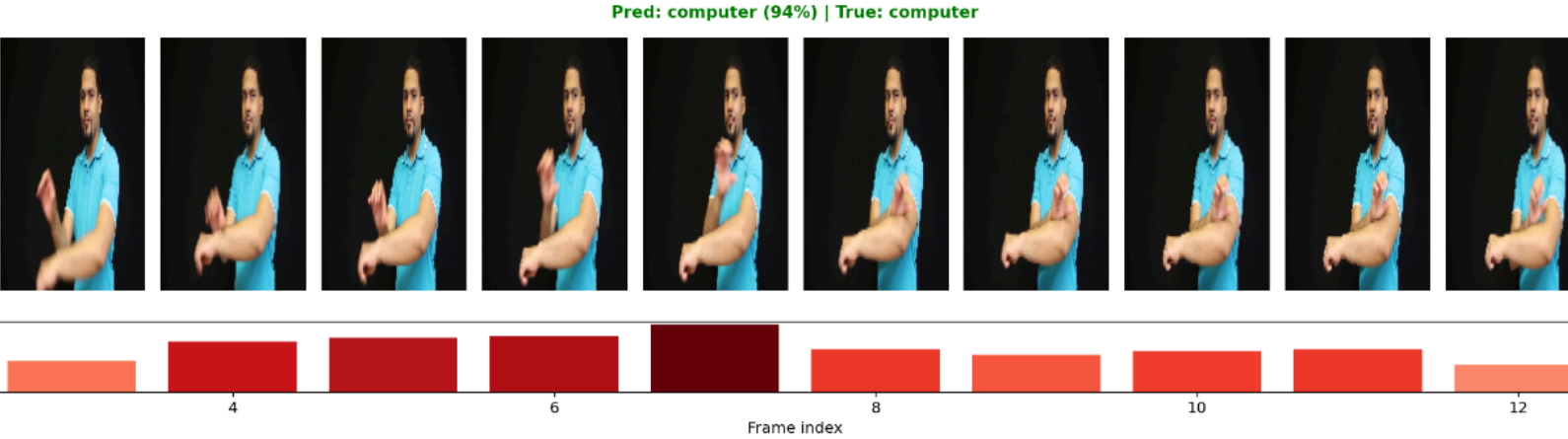

Figure 19: Attention on correctly classified "computer" (94% confidence). Weight concentrates on frames4–7 where the hand executes the sign's key motion over the opposite palm.

Across both correct and misclassified examples, the attention mechanism consistently focuses on a small window of frames rather than distributing weight uniformly, confirming it has learned to identify specific moments rather than averaging across the whole video. However, the crucial difference is what the peak lands on. On correctly classified examples, the peak aligns with the distinctive handshape. On misclassified examples, the peak still forms, but on a non-discriminative frame. This is consistent with the overfitting pattern discussed in Section 6.6.4: the attention layer is learning to find peaks, but the underlying BiLSTM feature representations still generalise poorly across unseen signers, so the peaks sometimes land on the wrong thing.

10.3 Aggregate Attention Distribution

To test whether the attention mechanism had learned a systematic temporal preference rather than attending idiosyncratically per video, we averaged the attention distributions across 100 test samples.

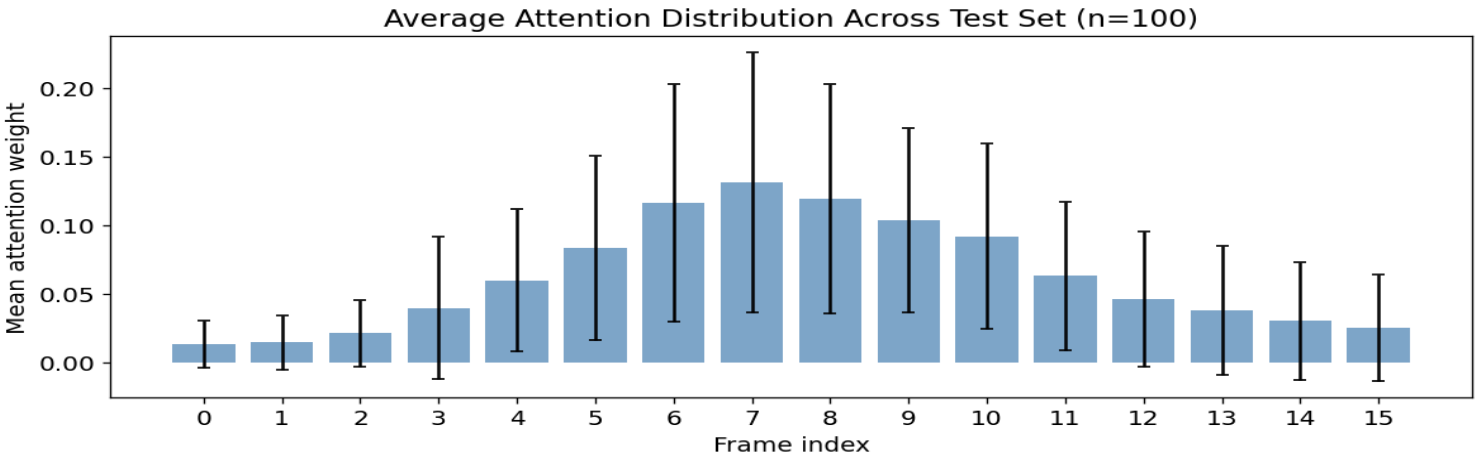

Figure 20: Mean attention weight per frame across 100 test samples. Error bars show one standard deviation.

The aggregated pattern reveals a clear structural bias. The model consistently assigns the highest weight to frames in the middle of the clip, with a peak at frame 7 (mean weight 0.131), and the lowest weight to the very first frame (mean weight 0.014). This is meaningful because WLASL videos typically begin with the signer in a neutral rest pose, with the distinctive sign executed in the middle of the clip. The fact that the attention mechanism independently discovered this temporal structure, without any explicit supervision on which frames to attend to, indicates it has learned genuinely interpretable temporal behaviour rather than arbitrary per-video patterns.

10.4 Implications

The attention analysis contributes three insights. First, it confirms that the attention mechanism is functioning as designed rather than collapsing to uniform weighting. Second, it complements Section 6.6.4's diagnosis of why Model C underperforms Model A. The attention layer itself is learning sensible temporal patterns, but the BiLSTM's 2.6 million parameters still overfit to signer-specific visual cues at the feature level, independent of the attention mechanism's behaviour. Third, the aggregate attention distribution could be repurposed as a diagnostic tool during training, which is a concrete direction for future work.

10. State-of-the-Art Comparison

Method	Dataset	Classes	Train Vids	Top-1
I3D (Li et al., 2020)	WLASL-100 full	100	2,000	65%
Video Transformer (Arnab et al., 2021)	WLASL-100 full	100	2,000	75%
Pose-TGCN (Li et al., 2020)	WLASL-100 full	100	2,000	55%
Ours (Model A)	WLASL-10 subset	10	111	38.89%

Table 18: State-of-the-art comparison. Direct comparison is not apples-to-apples due to different class counts and dataset sizes.

Published state-of-the-art methods on the WLASL benchmark achieve considerably higher accuracy than the current system. I3D (Li et al., 2020) achieved approximately 65% top-1 accuracy on WLASL-100 with around 2,000 training videos, and a Video Vision Transformer (ViViT) (Arnab et al., 2021) achieved approximately 75% on the same benchmark using long-range self-attention over spatiotemporal features. Pose-based graph convolutional network approaches achieved around 55% on the same setting.

However, this comparison is not directly equivalent. This is because methods such as I3D were pretrained on large-scale video datasets such as Kinetics before being applied to WLASL, while our models were built entirely from scratch. Additionally, state-of-the-art results were obtained using the full WLASL-100 training set of approximately 2,000 videos, whereas our team had access to only 864 videos as some original video links were no longer available at the time of download. These differences in pretraining scale, model architecture, and available training data largely account for the performance gap.

11. Conclusion and Future Work

Through this whole project, we managed to build a complete deep learning pipeline for ASL word classification from scratch, designing and testing five different model architectures across 10 systematic iterations.

For future work, training on the full WLASL-2000 dataset would give the BiLSTM and temporal attention models enough data to show their full potential. Switching from the current 20-sample validation set to k-fold cross-validation could also make model comparison much more reliable.

Additionally, adding video-level data augmentation techniques such as temporal dropout, speed perturbation, and temporal shifting could help to increase training data diversity without needing more labelled videos. We also could consider exploring a two-stream architecture that combines RGB pixel features and MediaPipe keypoint features as complementary inputs. We could also consider a partial CNN freezing strategy that preserves the low-level hand shape features learned during pretraining while allowing the later convolutional layers to fine-tune for the sign word classification task.

12. Reproducibility

12.1 Environment and Dependencies

This requires Python 3.10+ with CUDA 11.8. Trained the final model on an NVIDIA A100 GPU takes 15–30 minutes for 200 epochs with pre-extracted frames.

12.2 Setup

```
git clone https://github.com/Xephori/ADL.git
cd ADL
pip install -r requirements.txt
python data/preprocessing.py
```

Download pretrained weights from Google Drive and save all `.pth` files into `saved_models/`
https://drive.google.com/drive/folders/197ggC4-ApmuxbLZdNhSSSCJg2R8iSJeF?usp=drive_link

12.3 Reproducing the Best Result (Iteration 5, Model A)

```
python -m src.preextract_frames
python -m src.evaluate --model model_a \
  --weights saved_models/iter5_asl_pretrain_model_a_best.pth --split test
```

Expected output: Top-1 = 38.89%, Top-5 = 61.11%, Macro F1 = 0.34.

12.4 Training the Final Model from Scratch

```
python -m src.preextract_frames
python -m src.pretrain_cnn_asl --max-per-class 1000 --epochs 15
python -m src.train --model model_a \
  --pretrained-cnn saved_models/cnn_backbone_asl_pretrained.pth \
  --epochs 200 --lr 5e-4 --seed 42
python -m src.evaluate --model model_a \
  --weights saved_models/<checkpoint>_best.pth --split test
```

12.5 Running the Live Demo

With `iter5_asl_pretrain_model_a_best.pth` in `saved_models/` and frames pre-extracted, run:

```
python app.py
```

This opens a Gradio interface where a test video can be selected from the dropdown; clicking **Run Model** displays the 16 sampled frames alongside the top-5 predicted sign words with confidence scores.

13. References

Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M., & Schmid, C. (2021). ViViT: A Video Vision Transformer. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6836–6846.

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. In Proceedings of the International Conference on Learning Representations (ICLR).

Carreira, J., & Zisserman, A. (2017). Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6299–6308.

Koller, O., Camgoz, N. C., Ney, H., & Bowden, R. (2019). Weakly Supervised Learning with Side Information for Noisy Labeled Images. In Proceedings of the European Conference on Computer Vision (ECCV).

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., ... & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv preprint arXiv:1906.08172.

World Federation of the Deaf. (2023). Our Work. https://wfdeaf.org/our-work/